

A Labwork 5 Report

Luc Lacotte

0.1 Introduction

This labwork is about implementing a Gaussian blur filter and experimenting with shared memory.

0.2 Gaussian blur using CPU

First I implemented the Gaussian blur filter on the CPU. Here is the function:

```
def gaussian_blur_cpu(image):
    h, w, chan = image.shape
    output_image = np.zeros_like(image)
    image = image.astype(np.int32)

    for y in range(h):
        for x in range(w):
            for c in range(chan):
                out_of_range_pixel_value = 0
                pixel_value = 0
                for ky in range(-3, 4):
                    for kx in range(-3, 4):
                        ly = y + ky
                        lx = x + kx
                        if ly < 0 or ly >= h or lx < 0 or lx >= w:
                            out_of_range_pixel_value +=
                                GAUSSIAN_BLUR_MATRIX[ky + 3][kx + 3]
                        else:
                            pixel_value += image[ly, lx, c] *
                                GAUSSIAN_BLUR_MATRIX[ky + 3][kx + 3]
                output_image[y, x, c] = pixel_value // (
                    GAUSSIAN_BLUR_MATRIX_SUM - out_of_range_pixel_value)

    return output_image
```

This function is very slow on CPU, I first tried with big images and it took too much time that I stopped the process. For an image of 597x389, it took more than 6 seconds.

0.3 Gaussian blur using GPU

Then I implemented the same function but using, of course, the GPU. The kernel function is the same as the CPU function, but it uses the thread index to compute the pixel coordinates:

```
@cuda.jit
def gaussian_blur_gpu(src, dst, matrix, matrix_sum):
    tid_x = cuda.threadIdx.x + cuda.blockIdx.x * cuda.blockDim.x
    tid_y = cuda.threadIdx.y + cuda.blockIdx.y * cuda.blockDim.y
    h, w, chan = src.shape
    if tid_x >= w or tid_y >= h:
        return
    for c in range(chan):
        out_of_range_pixel_value = 0
        pixel_value = 0
        for ky in range(-3, 4):
            for kx in range(-3, 4):
                ly = tid_y + ky
                lx = tid_x + kx
                if ly < 0 or ly >= h or lx < 0 or lx >= w:
                    out_of_range_pixel_value += matrix[ky + 3, kx + 3]
                else:
                    pixel_value += src[ly, lx, c] * matrix[ky + 3, kx + 3]
        dst[tid_y, tid_x, c] = pixel_value // (matrix_sum -
            out_of_range_pixel_value)
```

0.4 Using shared memory

For using shared memory, I couldn't find how I was supposed to implement it, so Gen AI helped me a bit.

```
@cuda.jit
def gaussian_blur_gpu_shared_memory(src, dst, matrix, matrix_sum):
    tile = cuda.shared.array(shape=(32, 32, 3), dtype=numba.uint8)
    tx = cuda.threadIdx.x
    ty = cuda.threadIdx.y
    bx = cuda.blockIdx.x
    by = cuda.blockIdx.y
    bdx = cuda.blockDim.x
    bdy = cuda.blockDim.y
    x = bx * bdx + tx
    y = by * bdy + ty
    h, w, chan = src.shape
    if x >= w or y >= h:
        return
    for c in range(chan):
        tile[ty, tx, c] = src[y, x, c]
    cuda.syncthreads()
    for c in range(chan):
        out_of_range_pixel_value = 0
        pixel_value = 0
        for ky in range(-3, 4):
            for kx in range(-3, 4):
                ly = ty + ky
                lx = tx + kx
                if ly < 0 or ly >= bdy or lx < 0 or lx >= bdx or y + ky >= h or x + kx >= w:
                    out_of_range_pixel_value += matrix[ky + 3, kx + 3]
                else:
                    pixel_value += tile[ly, lx, c] * matrix[ky + 3, kx + 3]
        dst[y, x, c] = pixel_value // (matrix_sum - out_of_range_pixel_value)
```

0.5 Compare with and without shared memory

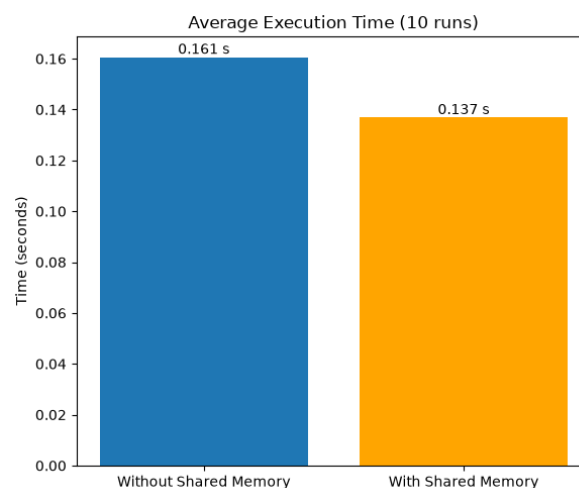


Figure 1: Execution times of the Gaussian blur filter on global memory and shared memory.

We can see that over some iterations, shared memory is a bit faster.